```haskell
-- CS 381 Homework1
-- Author: James Stallkamp

--Exercise 1 (Mini Logo)

--(a)
    data Cmd =
           Pen Mode
         | Moveto Pos Pos
         | Def Name [Pars] Cmd
         | Call Name [Vals]
         | Cmds [Cmd]

    data Mode = Up | Down
    data Pos =  I Num2 | S Name
    type Pars = Name
    type Vals =  Num2
    type Name = String
    type Num2 = Int
    type Cmds = [Cmd]

--(b)
{-
    def vector (x1, y1, x2, y2){
        pen up
        moveto (x1, y1)
        pen down
        moveto (x2, y2)
    }
-}
    x1 = 0
    y1 = 0
    x2 = 1
    y2 = 1
    vector :: Cmd
    vector = Def "vector" ["x1","y1","x2","y2"] (Cmds [Pen Up, Moveto (I x1) (I y1),
    Pen Down, Moveto (I x2) (I y2)])

--(c)
    steps :: Int -> Cmd
    steps 0 = Cmds [Pen Up, Moveto (I 0) (I 0)]
    steps x = Cmds [Pen Up, Moveto (I x) (I x), Pen Down, Moveto (I (pred x)) (I x),
              Pen Up, Moveto (I (pred x)) (I x), Pen Down, Moveto (I (pred x)) (I (pred
              x)), steps (pred x)]

-- Exercise 2 (Digital Circuit Design Language)
--(a)
data Circuit = Circon Gates Links
             | Lambda2
                deriving(Show)
data Gates = Gi Int GateFn Gates
             | Lambda
             deriving(Show)
data GateFn = And1
             |Or
             |Xor
             |Not
             deriving(Show)
data Links = FromTo (Int,Int) (Int,Int) Links
             | Lambda1
             deriving(Show)
--(b) Represent the half adder circuit in Haskell

halfadder = Circon (Gi (1) (Xor) (Gi (2) (And1) ((Lambda))))(FromTo(1,1)(2,1)
(FromTo(1,2)(2,2) (Lambda1)))

--(c)Define a Haskell function that implements a pretty printer for the above syntax

ppcir :: Circuit -> String
```

```haskell
ppcir (Lambda2) = ";"
ppcir (Circon a b)  = ppgate a ++" "++ pplinks b

ppgate :: Gates -> String
ppgate (Lambda) = ";"
ppgate (Gi c a b) = (show c) ++": "++ ppgatefn a++" " ++ ppgate b

ppgatefn :: GateFn -> String
ppgatefn (And1) = "and\n"
ppgatefn (Or) = "or\n"
ppgatefn (Xor) = "xor\n"
ppgatefn (Not) = "not\n"

pplinks :: Links -> String
pplinks (Lambda1) =";\n"
pplinks (FromTo (a,b) (c,d) e) = "from " ++ (show a) ++"."++ (show b) ++" to "++(show c) ++"."++ (show d)++" "++ pplinks e

nppcir :: Circuit ->IO()
nppcir  (x) = putStr (ppcir x)
-- Exercise 3 (Designing Abstract Syntax)
data Expr = N Int
            | Plus Expr Expr
            | Times Expr Expr
            | Neg Expr
            deriving (Show)
data Op = Add
        | Multiply
        | Negate
            deriving (Show)
data Exp = Num Int
            | Apply Op [Exp]
            deriving (Show)

--(a) Represent the expression -(3+4)*7 in the alternative abstract syntax.

expression = Apply Negate [(Num 7),(Apply Multiply [(Num 3),(Apply Add [Num 4])])]

--(b) What are the advantages or disadvantages of either representation?
-- Advantage: This representation always constructs the correct expr
-- Disadvantage: This representation does not always construct correct expression but can construct more "num Int"(s) than needed
--(c)
translate :: Expr -> Exp
translate (N a) = (Num a)
translate (Plus a b) = (Apply Add [translate a,translate b])

translate (Times a b) = (Apply Multiply [translate a,translate b])

translate (Neg a) = (Apply Negate [translate a])
```